Basics of Machine Learning and Artificial Intelligence

Introduction

Dr. Antonio Pellicani

A.Y. 2025/2026

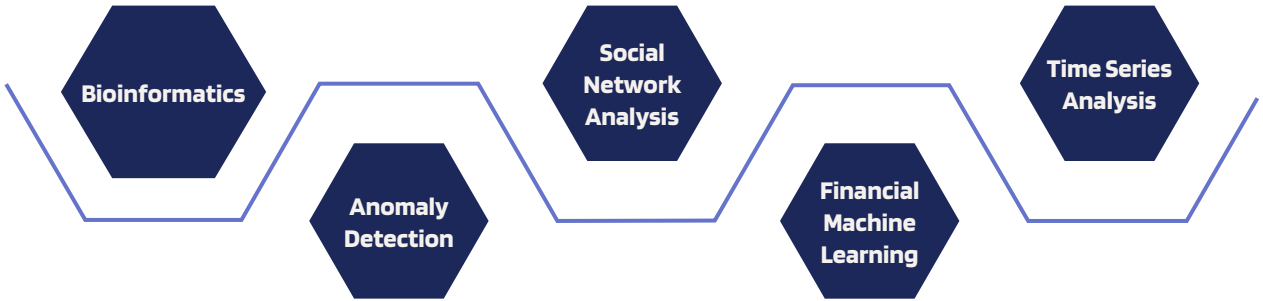
About me

My name is **Antonio Pellicani**.

- PhD in Computer Science and Mathematics in 2024
- Assistant Researcher @
  Knowledge Discovery and Data Engineering
  Research Group (KDDE), University of Bari "Aldo Moro"

KDDE

Research Interests:



About me

Where to find me:

- Room: Box 1, KDDE Lab, 4° Floor, Department of Computer Science, University of Bari "Aldo Moro"

- Office hours: Tuesday 9:00-11:30 (Please email to arrange meeting time.)

How to contact me:

- e-mail: antonio.pellicani@uniba.it

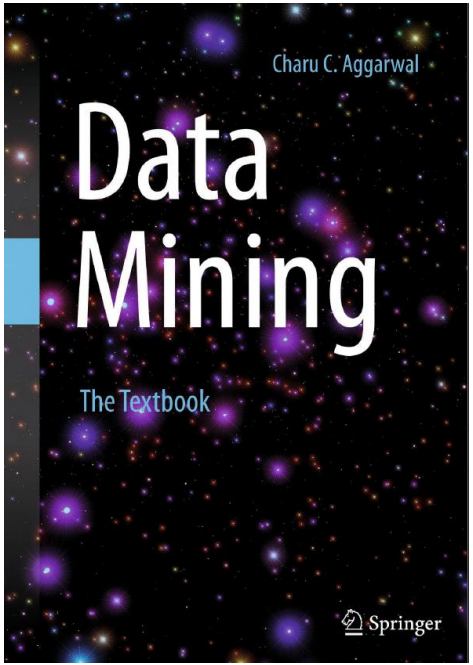
Basics of Machine Learning and Artificial Intelligence

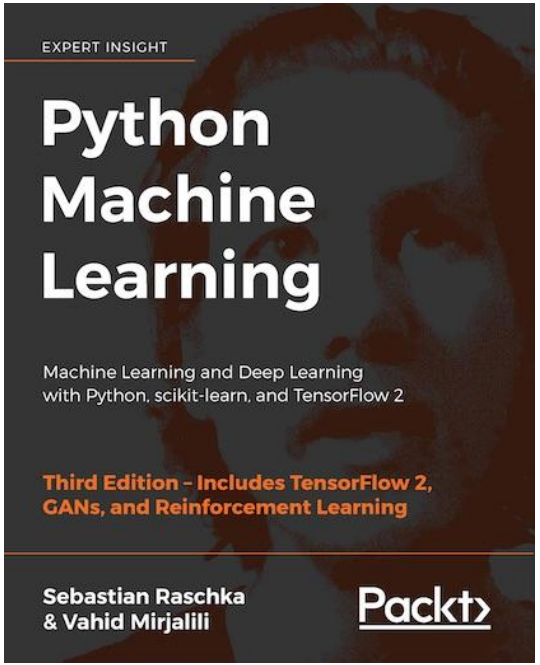
Course details

- Classes schedule:
  - Monday, 10:50AM – 1:30PM,
  - Tuesday, 2:30PM – 4:10PM.
- Slides and material will be available on my website: antoniopellicani.di.uniba.it/
- Slides password: BMLAI-2526

Basics of Machine Learning and Artificial Intelligence

Suggested books





Tentative course syllabus

01 What is Machine Learning

02 The CRISP-DM process

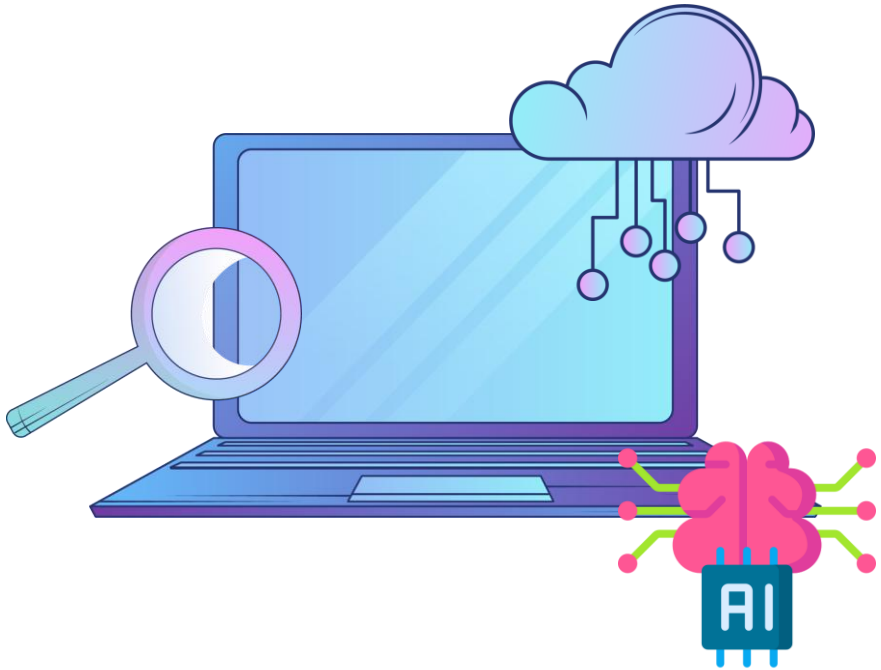
03 Python Language

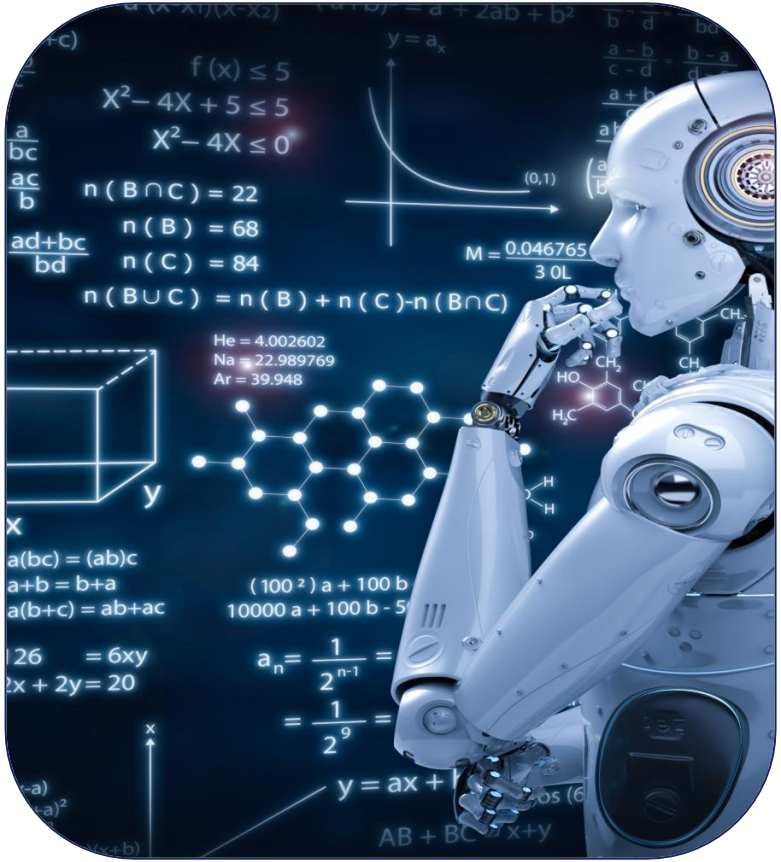
04 Classification models

05 Regression models

06 Cluster analysis

07 Neural networks





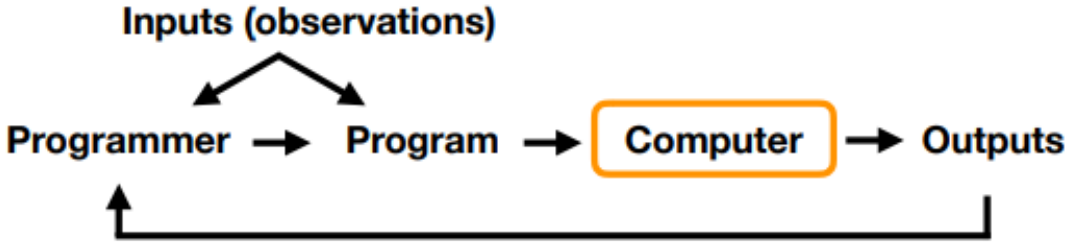
01

What is Machine Learning

"Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed"

Arthur Lee Samuel, AI pioneer, 1959

Traditional Programming Paradigm



Machine Learning

What is machine learning?

In traditional computer science **we write a program** that encodes a set of rules that are useful to solve a specific problem

- Sometimes it is very difficult to specify those rules (i.e. given a picture determine whether there is a cat in the image)

Learning systems are not directly programmed to solve a problem, instead **develop own program** based on:

- examples of how they should behave
- from trial-and-error experience trying to solve the problem

What is machine learning?

More formally, we want to implement an unknown function, only having access to sample input-output pairs (**training examples**).

- Learning simply means incorporating information from the training examples into the system

i.e. a function that takes a picture and returns "cat" or "not cat".
The problem is we do not know what that function looks like. We cannot write it down explicitly. It's unknown.

Our goal is to **approximate** it as closely as possible.

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Tom M Mitchell et al. "Machine learning. 1997"

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Tom M Mitchell et al. "Machine learning. 1997"

What does this means?

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Tom M Mitchell et al. "Machine learning. 1997"

Let's consider the problem of recognizing handwritten digit:

- Task *T*: ?
- Performance measure *P*: ?
- Training experience *E*: ?

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Tom M Mitchell et al. "Machine learning. 1997"

Let's consider the problem of recognizing handwritten digit:

- Task *T*: classifying handwritten digits from images
- Performance measure *P*: ?
- Training experience *E*: ?

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Tom M Mitchell et al. "Machine learning. 1997"

Let's consider the problem of recognizing handwritten digit:

- Task *T*: classifying handwritten digits from images
- Performance measure *P*: percentage of digits classified correctly
- Training experience *E*: ?

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Tom M Mitchell et al. "Machine learning. 1997"

Let's consider the problem of recognizing handwritten digit:

- Task *T*: classifying handwritten digits from images
- Performance measure *P*: percentage of digits classified correctly
- Training experience *E*: dataset of digits given classifications, e.g., MNIST

What makes a 5?

Why use learning?

It is very hard to write programs that solve problems like recognizing a handwritten digit:

- what distinguishes a 2 from a 7?
- how does our brain do it?

Instead of writing a program by hand, we collect examples that specify the correct output for a given input.
A machine learning algorithm then takes these examples and produces a program that does the job

- the program produced by the learning algorithm may look very different from a typical hand-written program. It **may contain millions of numbers**
- if we do it right, **the program works for new cases** as well as the ones we trained it on

Learning algorithms are useful in many tasks

- **Classification**: determine which discrete category the example is
- **Recognizing patterns**: speech Recognition, facial identity, etc.
- **Recommender Systems**: noisy data, commercial pay-off (e.g., Amazon,Netflix)
- **Information retrieval**: find documents or images with similar content
- **Computer vision**: detection, segmentation, depth estimation, optical flow, ...
- **Robotics**: perception, planning, ...
- **Recognizing anomalies**: unusual sequences of credit card transactions, panic situation at an airport
- **Spam filtering, fraud detection**: the enemy adapts so we must adapt too
- ...

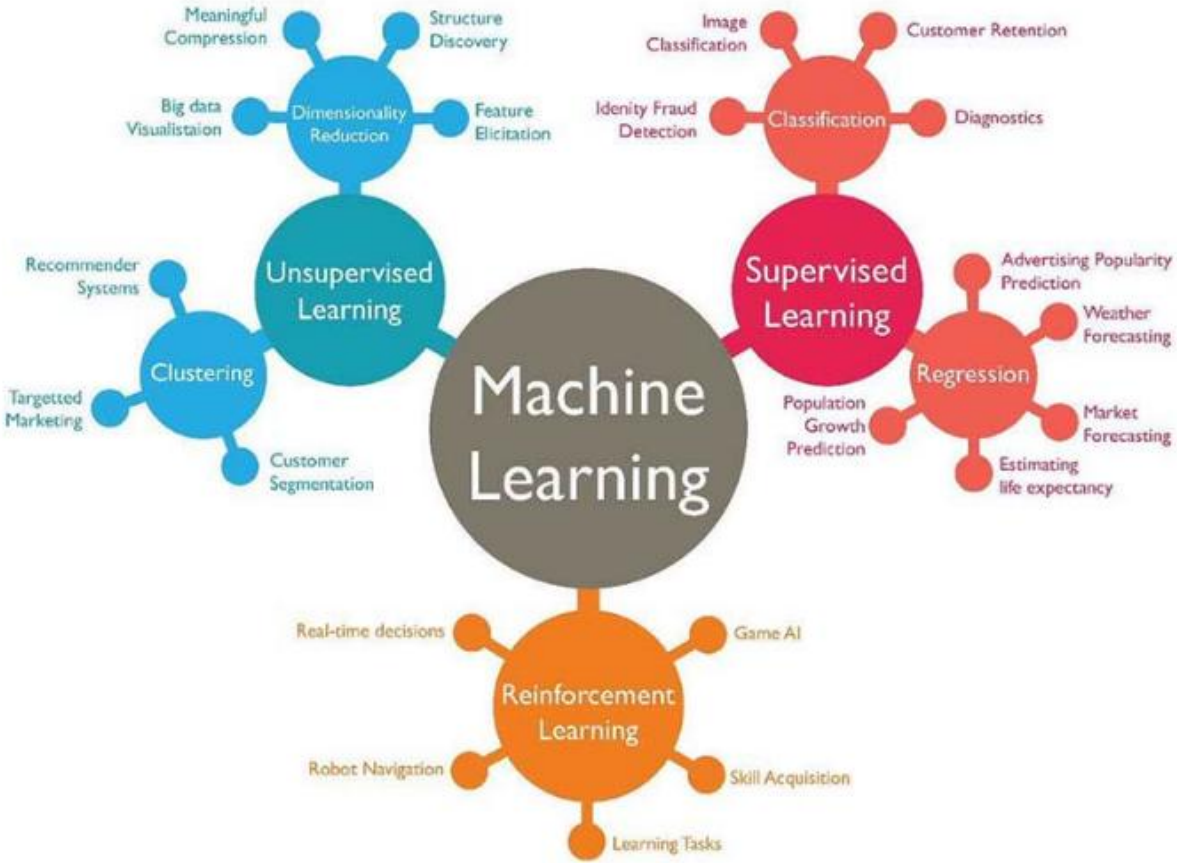
Machine Learning Paradigms

- **Supervised learning**: correct output known for each training example, learn to predict output when given an input vector
    - classification: 1-of-N output (speech recognition, object recognition, medical diagnosis)
    - regression: real-valued output (predicting market prices, customer rating)
- **Unsupervised learning**: create an internal representation of the input, capturing regularities/structure in data
    - discover patterns
    - summarizing the underlying relationship in the data

        Examples: form clusters; extract features
        - how do we know if a representation is good?

Machine Learning Paradigms

- **Reinforcement learning**: learn to take actions in an environment to maximize a cumulative reward signal
    - no labeled examples: feedback is only a scalar reward received after taking an action
    - agent observes a state, takes an action, receives a reward, transitions to a new state
    - Examples: game playing (Chess, Go), robot locomotion, self-driving cars


Machine Learning
Unsupervised Learning
Meaningful Compression
Structure Discovery
Dimensionality Reduction
Big data Visualisaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Image Classification
Customer Retention
Classification
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Market Forecasting
Estimating life expectancy
Population Growth Prediction
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

Types of Machine Learning Tasks

Machine learning (ML) tasks is usually divided into two main types

- predictive tasks / supervised learning
  - goal: learning a mapping from inputs x to outputs y, given a labeled set of input-output pairs $\mathcal{D} = \{(x_i, y_i\}^N$
    - $\mathcal{D}$ is called the **training set**, and N is the number of training examples
  - in the simplest setting, each training input x_i is a D-dimensional vector of numbers, called **features** or **attributes**
  - in general, x_i could be a complex structured object
    - image, sentence, email message, time series, molecular shape, …
  - similarly, the output variable can in principle be anything, but most methods assume y_i a categorical or nominal variable from some finite set, $y_i \in \{1, \dots, C\}$, or that y_i is a real-valued scalar
    - when y_i is **categorical**, the problem is known as **classification**, and
    - when y_i is **real-valued**, the problem is known as **regression**

Types of Machine Learning Tasks

- descriptive tasks / unsupervised learning
  - here we are only given inputs, $\mathcal{D} = \{(x_i\}^N$ and the goal is to find **interesting patterns** in the data
  - a much less well-defined problem, since we are not told what kinds of patterns to look for, and there is no obvious error metric to use
  - Association Analysis: discovers (the most interesting) patterns describing strongly associated features in the data/relationships among variables
    - Example: Market Basket Analysis ⇒ Discoverying interesting relationships among retail products.
  - Cluster Analysis: discovers groups of closely related facts/observations.
    - Example: Automaticaly grouping documents/web pages with respect to their main topic (e.g. sport, economy…)
  - Anomaly Detection: identifies facts/observations (Outlier/change/deviation detection) having characteristics significantly different from the rest of the data.

Machine Learning vs Data Mining

- Data-mining: historically using very simple machine learning techniques on very large databases because computers were too slow to do anything more interesting with ten billion examples
- Previously used in a negative sense: misguided statistical procedure of looking for all kinds of relationships in the data until finally find one
- Now lines are blurred: many ML problems involve tons of data
- But problems with AI flavor (e.g., recognition, robot navigation) still domain of ML

Machine Learning vs Statistics

- ML uses statistical theory to build models
- A lot of ML is rediscovery of things statisticians already knew; often disguised by **differences in terminology**
- But the emphasis is very different:
  - good piece of statistics: clever proof that relatively simple estimation procedure is asymptotically unbiased
  - good piece of ML: demo that a complicated algorithm produces impressive results on a specific task
- Can view ML as applying computational techniques to statistical problems. But go beyond typical statistics problems, with different aims (speed vs.accuracy)

Classification

The goal is to learn a mapping from inputs x to outputs y, where $y \in \{1, \ldots, C\}$, with C being the number of classes

- if $C = 2$, this is called binary classification
- if $C > 2$, this is called multiclass classification
- if the class labels are not mutually exclusive, we call it multi-label classification

One way to formalize the problem is as function approximation

- we assume $y = f(x)$ for some unknown function f
- the goal of learning is to estimate the function f given a labeled training set, and then to make predictions using $\hat{y} = \hat{f}(x)$

Main goal: making **predictions on novel inputs**, that we have not seen before (**generalization**), since predicting the response on the training set is easy

Probabilistic predictions

To handle ambiguous cases, it is desirable to return a **probability**

- we will denote the probability distribution over possible labels, given the input vector x and training set $\mathcal{D}$ by $p(y|x, \mathcal{D})$
  - this represents a vector of length C
  - if there are just two classes, it is sufficient to return the single number $p(y = 1|x, \mathcal{D})$, since $p(y = 1|x, \mathcal{D}) + p(y = 0|x, \mathcal{D}) = 1$

Given a probabilistic output, we can always compute our "best guess" as to the "true label" using:

$$\hat{y} = \hat{f}(x) = \operatorname{argmax}_{c=0}^{C} p(y = c|x, \mathcal{D})$$

Unsupervised Learning

The goal is to discover **interesting structure** in the data

- unlike supervised learning, we are not told what the desired output is for each input
- formalize the task as one of density estimation → build models of the form $p(\mathrm{x_i}\,|\theta)$

There are two differences from the supervised case:

- we have written $p(\mathrm{x_i}|\theta)$, instead of $p(\mathrm{y_i}|\mathrm{x_i}, \theta)$
  - supervised learning is conditional density estimation, whereas unsupervised learning is unconditional density estimation
  - $\mathrm{x_i}$ is a vector of features, so we need to create multivariate probability models
    - by contrast, in supervised learning, $\mathrm{y_i}$ is usually just a single variable that we are trying to predict
    - this means that for **most supervised learning problems, we can use univariate probability models**, which significantly simplifies the problem

Discovering clusters

A canonical example of unsupervised learning: clustering data into groups
Let K denote the number of clusters

- first goal: estimate the distribution over the number of clusters, $p(\mathrm{K}|\mathcal{D})$
  - we often approximate the distribution $p(\mathrm{K}|\mathcal{D})$ by its mode,

$$\mathrm{K}^* = \mathrm{argmax}_\mathrm{k}\ p(\mathrm{K}|\mathcal{D})$$

- second goal: estimate which cluster each point belongs to
  - let $\mathrm{z_i} \in \{1, \ldots, K\}$ represent the cluster to which data point i is assigned
  - We can infer which cluster each data point belongs to by computing

$$\mathrm{z_i}^* = \mathrm{argmax}_\mathrm{k}\ p(\mathrm{z_i} = \mathrm{K}|\mathrm{x_i}, \mathcal{D})$$

Basics Concepts

Parametric vs non-parametric models

We will focus on probabilistic models of the form

$$p(y|x, \mathcal{D}) \text{ or } p(x)$$

depending on being interested in supervised or unsupervised learning resp.
When a model has a **fixed number of parameters**, it is called parametric
When the number **of parameters grows with the amount of training data**, it is called non-parametric.

- parametric models
  - advantage: being faster to use
  - disadvantage: making stronger assumptions about the nature of the data distributions
- non-parametric models
  - advantage: are more flexible
  - disadvantage: often computationally intractable for large datasets

Basics Concepts

K-nearest neighbors: a simple non-parametric classifier

K nearest neighbor (KNN) classifier

- looks at the K points in the training set that are nearest to the test input x,
- counts how many members of each class are in this set,
- returns that empirical fraction as the estimate:

$$p(\mathrm{y} = \mathrm{c}|x, \mathcal{D}, \mathrm{K}) = \frac{1}{\mathrm{K}} \sum_{\mathrm{i} \in \mathrm{N_K}(\mathrm{x}, \mathcal{D})} \mathbb{1}(\mathrm{y_i} = \mathrm{c})$$

where:

- $\mathrm{N_K}(x, \mathcal{D})$ are the (indices of the) K nearest points to x in $\mathcal{D}$
- $\mathbb{1}(\mathrm{e})$ is the indicator function, returning 1 when e is true, and 0 otherwise

Basics Concepts

Linear regression: a parametric model

One of the most widely used models for regression: linear regression

- the response is a linear function of the inputs

$$y(x) = w^{T}x + \epsilon = \sum_{j=1}^{D} w_j x_j + \epsilon$$

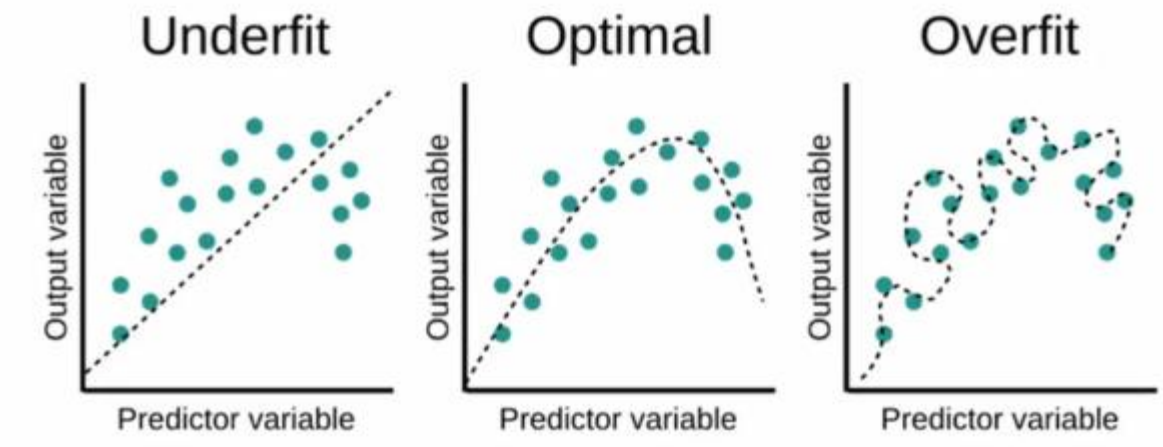
where ϵ is the residual error between our linear predictions and the true response

Basics Concepts

Overfitting

When we fit highly flexible models, we need to be careful that we do not overfit the data

- that is, we should avoid trying to model every minor variation in the input,
- this is more likely to be noise than true signal.



Basics Concepts

Model Selection

When we have a variety of models that solve the same problem but with different complexity, how should we pick the right one?

Basics Concepts

Model Selection

When we have a variety of models that solve the same problem but with different complexity, how should we pick the right one?

- a natural approach is to compute the **misclassification rate** on the training set for each method:

$$\mathrm{err}(\mathrm{f}, \mathcal{D}) = \frac{1}{\mathrm{N}} \sum_{\mathrm{i}=\mathrm{i}}^{\mathrm{N}} \mathbb{1}(\mathrm{f}(\mathrm{x_i}) \neq \mathrm{y_i})$$

Where f is the classifier.

- however, what we care about is **generalization error**, which is the expected value of the misclassification rate when averaged over future data
  - approximated by computing the misclassification rate on a large independent test set, not used during model training

Basics Concepts

Model Selection

Unfortunately, when training the model, we don't have access to the test set

- we can create a test set by partitioning the training set into two: the part used for training the model, and a second part, called the **validation set**, used for selecting the model complexity
  - fit all the models on the training set, and evaluate their performance on the validation set, and pick the best
  - once picked the best, refit it to all the available data
  - often 80% of the data for the training set, and 20% for the validation

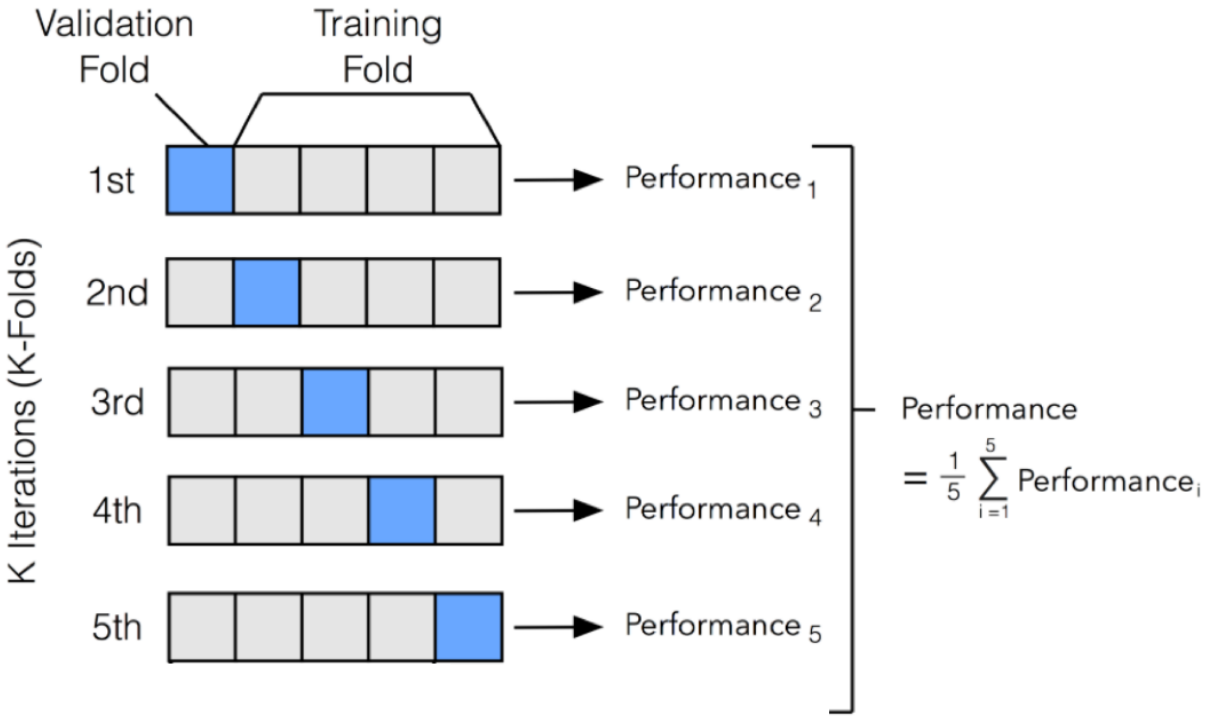
Basics Concepts

Model Selection

- a better solution to this is to use **cross validation** (CV)
  - split the training data into K folds
  - for each fold $k \in \{1, \ldots, K\}$, train on all the folds but the k-th, and test on the k-th, in a round-robin fashion
  - compute the error averaged over all the folds, and use this as a proxy for the test error
- If we set $K = N$, then we get a method called **leave-one out cross validation** (LOOCV)

Basics Concepts

Model Selection



Basics Concepts

No free lunch theorem

All models are wrong, but some models are useful.

George Box, 1987

- much of machine learning is concerned with devising different models, and different algorithms to fit them
- there is **no universally best** model. This is also known as *the no free lunch theorem*
- the reason for this is that a set of assumptions that works well in one domain may work poorly in another

Resources

- Murphy, Machine Learning: a Probabilistic Perspective, 2012, Chap.1 – Introduction.
- Course material for STAT 479: Machine Learning (FS 2019) taught by Sebastian Raschka at University Wisconsin-Madison- Lecture 1: What is Machine Learning? An Overview. https://sebastianraschka.com/teaching/stat479-fs2019/